

Quick Start Guide — Decoder

Get a DCC decoder running in minutes

TI MSPM0G3507 LaunchPad + Code Composer Studio / Theia edition. OpenDccCLib itself runs on any microcontroller with a C compiler and an edge interrupt; this guide covers one concrete path.

Generated 2026-09-23 from the repository sources by documentation/guides/build_pdfs.py

Table of Contents

1. What is a DCC Decoder?	3
1.1 Key Terms	3
2. What You Need	4
2.1 Hardware	4
2.2 Signal Pins	4
2.3 Software	4
3. Project Setup	5
4. Understanding dcc_user_config.h	6
5. Building and Flashing	7
5.1 What Happens at Power-On	7
6. How the Decoder Works	8
7. Customizing Callbacks	9
7.1 Terminal Commands	9
8. What's Next	10

1. What is a DCC Decoder?

A DCC decoder sits in a locomotive or an accessory, listens to the DCC signal on the rails, recognizes the commands addressed to it, and drives motors, lights, servos, solenoids or sound.

OpenDccCLib does the protocol work: it classifies bits from edge timing, assembles and validates packets, matches addresses against the CVs, and dispatches commands to your callbacks. You write the code that moves the hardware.

1.1 Key Terms

- **Bit decoder:** tells one-bits (58 μ s halves) from zero-bits (100 μ s or longer) by edge timing.
- **Packet decoder:** assembles bytes, checks the XOR byte, matches the address, dispatches.
- **Configuration Variable (CV):** a numbered setting stored in the decoder: address, speed curve, function mapping.
- **Service mode:** programming-track mode; the decoder answers a successful command with a 6 ms current pulse (ACK).
- **Fail-safe:** with no packet addressed to it for CV 11 $\times$ 100 ms, the decoder stops its outputs.

2. What You Need

2.1 Hardware

- TI MSPM0G3507 LaunchPad (LP-MSPM0G3507)
- A DCC source: a commercial command station through a level converter, or a second LaunchPad running the command-station example
- For a real track: an input circuit that turns track-level DCC into 3.3 V logic (optocoupler or divider)
- A USB cable; optionally a motor driver, LEDs or servos to see the outputs

2.2 Signal Pins

All signal pins are on port B; they come from the project's SysConfig file.

LaunchPad pin	Function
PB1	Main-track DCC in, edge interrupt; from command station PB1
PB4	Service-track DCC in, edge interrupt; from command station PB4
PB12	ACK out; drives the current load the command station senses; to CS PB12
PB17	Track-select in; from CS PB17
PB3	Test pulse on each accepted edge (logic-analyzer aid)
PB22	Heartbeat LED
PA10 / PA11	UART TX / RX to the on-board debug probe, 230400 8N1

Two-board setup: CS PB1 to decoder PB1, CS PB4 to decoder PB4, decoder PB12 to CS PB12, CS PB17 to decoder PB17, and GND to GND.

2.3 Software

- TI Code Composer Studio or CCS Theia, with SysConfig
- A serial terminal at **230400 baud**, 8N1
- The OpenDccCLib source tree

3. Project Setup

The example is at `applications/ti_theia/mspm03507_launchpad/decoder/`. Import it in CCS (File > Import > CCS Projects); it reaches the library through the `dcc_lib` symlink to `src/dcc/`.

```
decoder/
  decoder.c                <- main entry: config struct, edge ISR, ring buffer, main loop
  dcc_user_config.h        <- role flags and limits
  decoder_command_parser.c/h <- UART command-line interface
  application_callbacks/
    callbacks_dcc.c/h      <- CV storage stub, CV29 feature mask, command callbacks
  application_drivers/
    ti_driverlib_dcc_driver.c/h <- timestamp, interrupt mask, RailCom delay
    ti_driverlib_uart_driver.c/h <- UART I/O
    ack_pulse_driver.c/h    <- ACK current load
  dcc_lib/                 <- the library (symlink; do not edit)
  Debug/ti_msp_dl_config.c/h <- SysConfig-generated init
```

4. Understanding dcc_user_config.h

```
#define DCC_COMPILE_DECODER
#define DCC_COMPILE_RAILCOM // RailCom transmit engine (inert until wired)
// #define DCC_COMPILE_ACCESSORY_DECODER

#define USER_DEFINED_DCC_DECODER_MAX_FUNCTIONS 29 // F0-F28
#define USER_DEFINED_DCC_DECODER_PACKET_QUEUE_DEPTH 8 // packets waiting for main-loop dispatch
```

Do not define DCC_COMPILE_COMMAND_STATION in a decoder project unless the device really is both, as a booster or repeater would be.

5. Building and Flashing

1. Build (Project > Build, or Ctrl+B).
2. Connect the LaunchPad over USB and flash (Run > Debug, or F11).
3. Open a terminal at 230400 baud. The banner reads DCC Decoder - MSPM0G3507 LaunchPad.

5.1 What Happens at Power-On

`main()` initializes the drivers and the CV defaults (short address 3: CV 1 = 3, CV 29 = 0x06), hands the config struct to `DccConfig_initialize()`, which reads the addressing CVs, then loops. The decoder listens on PB1 immediately.

```
while (1) {  
    _drain_edge_buffer();           // ISR timestamps -> bit decoder  
    DccConfig_run();               // packet dispatch, callbacks, ACK timing, fail-safe  
    CallbacksDcc_drain();         // RECV log lines -> UART  
    TI_UartDriver_echo_process();  
    DecoderCommandParser_process();  
}
```

6. How the Decoder Works

- **Edge capture (interrupt).** Every edge on the selected input stores a microsecond timestamp in a 256-entry ring. The interrupt does nothing else, so it stays well under the 58 μ s half-bit.
- **Bit decoding (main loop).** The ring is drained into `DccConfig_decoder_edge_isr()`. An interval under 80 μ s is a one-bit half, 80 μ s or more a zero-bit half.
- **Packet decoding.** Ten or more one-bits arm the preamble; bytes follow until the end bit. The XOR is checked, the address matched against CV 1 or CV 17–18 as CV 29 bit 5 selects, and the packet is queued.
- **Dispatch.** `DccConfig_run()` takes queued packets and calls your callbacks, so they never run inside an interrupt.

Every decoded command is echoed on the terminal as a RECV line, for example RECV SPEED addr=3 speed=64 dir=FWD mode=128.

7. Customizing Callbacks

All application logic lives in `callbacks_dcc.c`. The demo logs to the UART; replace the bodies with your motor, light and servo control.

```
void CallbacksDcc_on_speed_command(uint16_t address, uint8_t speed,
                                   bool direction, dcc_speed_mode_enum mode) {
    /* speed: 0 = stop, 1 = emergency stop, 2.. = steps; direction: true = forward */
    set_motor_speed(speed, direction);
}

void CallbacksDcc_on_function_command(uint16_t address, uint8_t function_number, bool state) {
    if (function_number == 0) set_headlight(state); /* F0 = headlight */
}
```

Callback	Fires on
<code>on_speed_command</code>	Speed and direction for this address
<code>on_emergency_stop_command</code>	Emergency stop, addressed or broadcast
<code>on_function_command</code>	Function on or off, F0–F68
<code>on_accessory_basic_command</code> , <code>on_accessory_extended_command</code>	Turnout, signal aspect
<code>on_cv_write_command</code> , <code>on_cv_verify_command</code> , <code>on_cv_bit_command</code>	CV access; the <code>service_mode</code> flag tells programming track from main track
<code>on_consist_command</code>	Consist set or clear
<code>on_binary_state_short_command</code> , <code>on_binary_state_long_command</code> , <code>on_analog_function_command</code>	Feature-expansion commands
<code>on_failsafe_entered</code> , <code>on_failsafe_exited</code>	No addressed packet within CV 11 × 100 ms; packets resumed
<code>cv29_apply_supported_features</code>	Required: clear the CV 29 feature bits this product does not implement

7.1 Terminal Commands

`ADDR <n> <SHORT|LONG|ACC|ACCE>` sets the address, `CLEAR` flushes the log, `ACK`, `ACK ON|OFF`, `ACK <width_us>` and `ACK TEST [count]` exercise the ACK pulse, `STATUS` shows the address, `HELP` lists everything.

8. What's Next

- **RailCom replies.** Add a current-source circuit on a GPIO, set `railcom_tx_pin_set` and a cycle-accurate `railcom_delay_us`, and answer Channel 2 requests in `on_railcom_request`. The library transmits the address broadcast in Channel 1 on its own.
- **Persistent CVs.** Replace the RAM stub behind `cv_read` / `cv_write` with Flash or EEPROM so settings survive a power cycle.
- **Another MCU.** Copy the project and rewrite the driver files. The only interrupt requirement is a microsecond timestamp on each DCC edge.
- **Developer Guide.** The companion Developer Guide, Decoder, covers the config struct, bit and packet decoders, CV storage, service mode, RailCom replies and fail-safe in depth.